

JJ リファレンス

モデル

Jujutsu リポジトリは DAG(有向非巡回グラフ)で、そのノードは **チェンジ** と呼ばれます。それぞれのチェンジは以下のものを持っています:

- リポジトリディレクトリ内のファイルシステムの状態。それぞれのチェンジがディレクトリとすべてのファイルの完全なコピーを保持していることを想像してもよいですが、実際には、**jj** はもっと効率的にそれを行っています。
- ファイルの**衝突**。チェンジ内のファイルには様々な理由で衝突があるかもしれません。それらの衝突は そのチェンジにローカルなものです。(git と異なり、衝突は **jj** を使うことを妨げません。)
- 一つかそれ以上の**親**のチェンジ。ですが、親がなく常に空ディレクトリを持つルート¹のチェンジもあります。
- テキストによるチェンジの説明で、コミットメッセージとも呼ばれます。これは常に存在しますが、デフォルトは空文字列です。

いくつかの付加情報が DAG に添えられています:

- チェンジのうち一つだけが**作業中のチェンジ**で、@で表されています。この文書ではそれを“作業コピーのリビジョン”と呼びます。(これは git の HEAD に似ています。)
- チェンジには単一な文字列のラベル **ブックマーク**があるかもしれません。(git と連携する場合、ブックマークはブランチ名として機能します。)
- リポジトリは(Github のような)**リモートリポジトリ**と関連付けられているかもしれません。その場合、push または fetch 時に、**jj** はそれぞれのブックマークの**最後に知っている位置**を **ブックマーク @リモート**で表して記録します(例: feat-ui@origin)。

ほとんどの **jj** のコマンドはローカルリポジトリの DAG を何らかの方法で修正します。いくつかの一般的なルールが、リポジトリがどのように変化するかを予測する手助けになるでしょう:

- あるチェンジを@で指定すると、リポジトリディレクトリはそのチェンジのファイルと一致するように更新されます。
- @が指しているチェンジを削除した場合、@はその親から作られた新しい空のチェンジに移動します。
- チェンジにファイルの修正と説明がなく、@やブックマークから参照されていない場合、静かに闇へと消えていきます。
- チェンジは差分を表しています。チェンジを移動すると新しい親に差分を適用しようとします。これはマージコンフリクトを起こすかもしれません。
- 多くのコマンドはデフォルトでは@に作用します。ほぼ全てのコマンドは他のチェンジに作用させるために **-r/--revision** の引数を取ることができます。

ファイルの衝突

作業中のチェンジ(@)に**ファイルの衝突**がある場合、単に衝突マーカー(<<<<<<<, ===== など)が無くなるようにそのファイルを編集すれば解決されます。バイナリファイルの場合は、あるべきバージョンのファイルに置き換えてください。この目的には **jj restore** が便利かもしれません。(git と異なり、ファイルの衝突は作業を妨げません。)

jj git push

jj git push はチェンジをローカルリポジトリからリモートリポジトリにコピーします。ローカルのチェンジが最後のプッシュ時から修正されている場合、それはリモートリポジトリの新しいチェンジになります(git の強制プッシュのように以前のコミットを新しいコミットで置き換えます)。この操作をうっかり main に行くことを防ぐため、**jj git push** は主要なブランチにプッシュされた全てのチェンジを変更不能にします。これらのチェンジは必要ならば引き続き

編集できますが、それには **--ignore-immutable** フラグを与える必要があります。

すべてのローカルブックマークは同様にリモートリポジトリにコピーされます。ブックマークがローカルとリモートの両方にある場合、**jj** はその(ローカルに記録された)**最後に見ていた位置**がリモートリポジトリの現在の位置と一致するかを調べます。一致する場合、リモートリポジトリのブックマーク位置は更新されます。そうでなければこのコマンドは失敗し、(あなたが最後にプッシュしてから誰かがブックマークを更新したということなので)まずは **jj git fetch** を行うよう知らせます。

jj git fetch

jj git fetch はリモートリポジトリのチェンジをローカルリポジトリにコピーします。チェンジがリモートリポジトリで修正された場合、それはローカルでも新しいチェンジに変わります。ですが、ほとんどの場合は単に新しいチェンジを取得します。

ローカルのブックマークはリモートリポジトリにあるチェンジと一致するように進みます。しかし、リモート上のブックマークのチェンジがローカルのチェンジの子孫でない場合、**jj git fetch** はそのブックマークのもう一つのコピーを作ります。これはブックマーク名は単一であるというルールを破るので、**ブックマークの衝突**と呼ばれます。(git pull がマージコンフリクトを起こすことに似ています。)この“ブックマークの衝突”をどう解決するかはあなた次第です。それにはいくつかの選択肢があります:

- 二つのチェンジをマージしたい場合、**jj new チェンジ ID-1 チェンジ ID-2**を行い、衝突を解決し、その後 **jj bookmark move ブックマーク名**でブックマークを更新します。(チェンジ ID は **jj bookmark list ブックマーク名**を実行すると取得できます。)
- 二つのチェンジのうち一つを捨てて単にもう一方を使いたい場合、残したいチェンジに対し **jj bookmark move ブックマーク名 -t チェンジ ID**を行います。
- 一方のチェンジがもう一方のチェンジの**後**に来るようリベースしたい場合、**jj rebase -b チェンジ ID-2 -d チェンジ ID-1**を行い、その後 **jj bookmark move ブックマーク名 -t チェンジ ID-2**を行います。これは二つ目のチェンジのみではなく、一つ目のチェンジから分岐したすべてのチェンジをリベースします。

コマンド

共通設定のコマンド

```
jj config set --user user.name 自分の名前
jj config set --user user.email 自分のEMAIL
jj config set --user ui.editor 自分のエディター
```

```
jj config edit --user // 設定ファイルを手で編集する
```

--user の代わりに **--repo** を渡すと、リポジトリ固有の優先的な設定を変更できます。

リポジトリのコマンド

- jj git init**、または **jj git clone URL [送り先]**。git バックエンドのリポジトリを作成またはクローンする。
- jj git init --colocate**。既存の git リポジトリが **jj** リポジトリにもなるようにする。

ローカルリポジトリを編集する

添付の JJ チートシートでは **jj** リポジトリを編集するためのもっとも一般的で基本的なコマンドを視覚的に表します。

他の **jj** コマンドの組み合わせとして考えるとよい、いくつかの“エイリアス”コマンドもあります:

- jj commit**。 **jj describe**; **jj new** の省略形。
- jj bookmark set ブックマーク名**。ブックマークの **create** または **move**、どちらか有効なほう。

JJ チートシート

jj

リポジトリ内の重要な
チェンジを表示する。

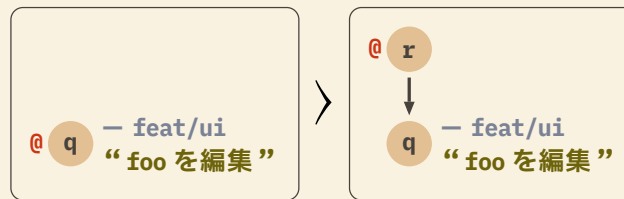
jj undo

最後のコマンドを
アンドゥする。

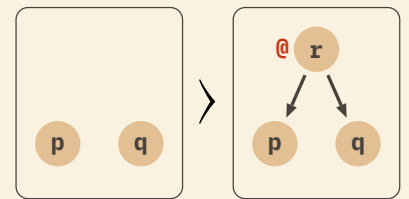
jj status

現在のチェンジと親の
チェンジ、ファイルの
修正を表示する。

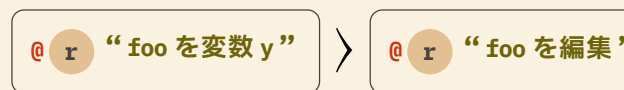
jj new



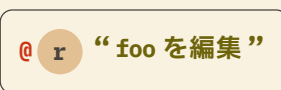
jj new p q



jj describe -m "foo を編集"

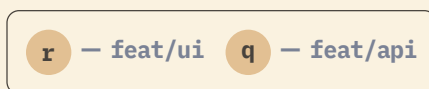


jj show



このチェンジの説明を表示する。

jj bookmark list



すべてのブックマークを表示する。

jj bookmark create feat/ui



jj bookmark delete feat/ui



jj bookmark move feat/ui



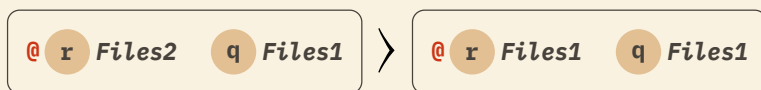
jj bookmark rename feat/ui feat/ux



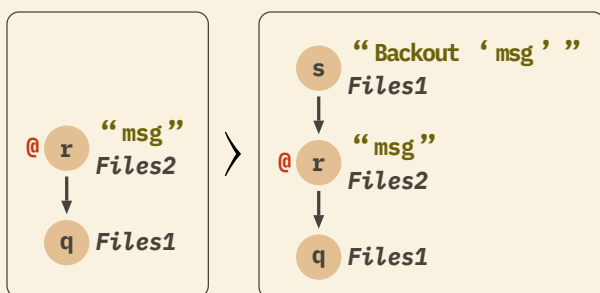
jj edit q



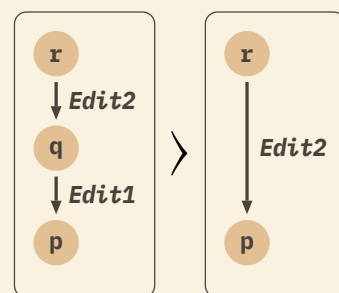
jj restore --from q (paths..)



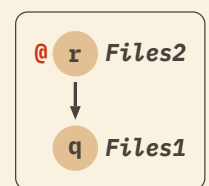
jj backout



jj abandon q

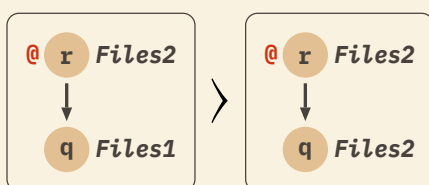


jj diff (paths..)



Files1 と Files2 の
間の差分を表示する。

jj squash



r — チェンジ

@ — 作業中のチェンジ("作業コピーのリビジョン")

"foo を編集" — チェンジの説明

feat/ui — ブックマーク

Files1 — ファイルシステムの状態

Edit1 — 二つのチェンジの間の差分

justinpombrio.net

& lark.gay

Feb 2025

日本語訳: <https://github.com/ygohko/jj-notes>